

PHASE 11

SECURITY

High-End System Design Interview Notes

THE BIG IDEA

Security is not one feature. It is a set of controls protecting identity, permissions, data, communication, APIs, secrets, and the system itself.

Authenticate → Authorize → Encrypt → Validate → Limit → Monitor → Protect

Phase 11 Learning Map

1. Authentication
2. Authorization
3. RBAC
4. JWT and session authentication
5. OAuth 2.0 concepts
6. TLS
7. Encryption at rest and in transit
8. Password hashing
9. API keys and secrets
10. Rate limiting
11. SQL injection
12. XSS
13. CSRF
14. Secrets management
15. Defense in depth and security interview framework

INTERVIEW MINDSET

Do not say “the API is secure because it uses JWT.” Security requires multiple independent controls. Explain what each control protects and what happens if another layer fails.

1. Authentication — Who Are You?

Authentication (AuthN) verifies the identity of a user, service, or client.

- Username/password.
- One-time codes or multi-factor authentication.
- Passkeys or other strong authenticators.
- Access tokens issued after successful authentication.
- Service-to-service credentials.

```
Client → Login Credentials → Identity System
      ↓
    Identity verified
      ↓
    Token / Session
```

MEMORY

Authentication = “Who are you?”

2. Authorization — What Are You Allowed To Do?

Authorization (AuthZ) decides whether an authenticated identity is allowed to perform a particular action on a resource.

- User is authenticated.
- System identifies the requested resource and action.
- Policy checks whether the user has permission.
- Request is allowed or denied.

```
User → authenticated ✓
      → DELETE /users/42
      → permission check
      → allow / deny
```

MEMORY

Authorization = “What are you allowed to do?”

3. RBAC — Role-Based Access Control

RBAC assigns permissions to roles, then assigns users to those roles.

```
User → Role → Permissions
```

```
Harshith → ADMIN → create/read/update/delete
```

```
Student → STUDENT → read own attendance
Teacher → TEACHER → manage class attendance
```

- Roles simplify permission management.
- A user can have one or multiple roles depending on the system.
- Permissions should still be checked at the resource/action level where required.

INTERVIEW TIP

Avoid putting “if user == admin” everywhere. Centralize authorization policies and make permissions explicit.

4. JWT Authentication

A JSON Web Token (JWT) is a signed token that can carry claims about an identity or authorization context.

```
JWT = Header.Payload.Signature
```

- Header describes token metadata such as the signing algorithm.
- Payload contains claims such as subject, issuer, audience, and expiry.
- Signature helps the recipient detect unauthorized modification.
- JWTs are commonly sent in an Authorization header.

```
Authorization: Bearer <access-token>
```

IMPORTANT

A signed JWT is not automatically encrypted. Do not put secrets or sensitive information in a normal JWT payload simply because it is signed.

5. Session-Based Authentication

With session authentication, the server maintains session state and the browser typically sends a session identifier with later requests.

```
Login → Server creates session
      → Session ID stored in secure cookie
      → Browser sends cookie on later requests
      → Server looks up session
```

- Session data can live in a database or shared cache.
- Cookie security flags matter.
- Distributed applications need shared session storage or another strategy if sessions are not local.

6. JWT vs Session

Aspect	JWT	Session
State	Token carries claims; server may not need session lookup	Server maintains session state
Revocation	Can be more complex for long-lived tokens	Server can invalidate a session centrally
Scaling	Often convenient for stateless APIs	Needs shared session store for multiple instances
Storage/security	Protect token from theft	Protect session cookie and session store

NO UNIVERSAL WINNER

Choose based on revocation needs, client type, architecture, token lifetime, operational requirements, and security model.

7. OAuth 2.0 Concepts

OAuth 2.0 is an authorization framework that allows a client to obtain access to protected resources without directly handling the resource owner's password.

- Resource owner — the user who owns the data.
- Client — application requesting access.
- Authorization server — issues authorization credentials/tokens.
- Resource server — hosts protected resources.
- Access token — credential used to access a resource.

```

User → Authorization Server
    ↓ consent / authentication
Access Token
    ↓
Client → Resource Server
  
```

IMPORTANT

OAuth 2.0 is primarily an authorization framework. OpenID Connect (OIDC) builds an authentication/identity layer on top of OAuth 2.0.

8. TLS

TLS protects data in transit between communicating endpoints by providing confidentiality, integrity, and server authentication through certificates.

- Confidentiality — outsiders should not read protected traffic.
- Integrity — modifications should be detectable.
- Authentication — certificates help the client verify the server identity.

HTTPS = HTTP + TLS

INTERVIEW FLOW

Client connects → TLS handshake → certificate validation → secure keys established → encrypted HTTP traffic.

9. Encryption at Rest vs In Transit

Type	Protects	Examples
In transit	Data moving between systems	HTTPS/TLS, secure service-to-service connections
At rest	Stored data	Encrypted disks, encrypted database/storage, encrypted backups

MEMORY

In transit = moving. At rest = stored. A secure system often needs both.

10. Password Hashing

Passwords should not be stored as plaintext. Password hashing uses a deliberately slow password-hashing function designed to make guessing attacks expensive.

- Use password-hashing algorithms designed for passwords, such as Argon2id, bcrypt, or scrypt.
- Use a unique salt for each password.
- Never store plaintext passwords.
- Do not use fast general-purpose hashes such as plain SHA-256 as a password storage scheme.

Password → Password Hashing Function + Salt → Stored Hash

Login Password → Verify Against Stored Hash

IMPORTANT

Encryption is reversible with a key; password hashing is designed to be one-way. For passwords, use a password-specific hashing algorithm.

11. API Keys & Secrets

API keys, database passwords, signing keys, cloud credentials, and similar values are secrets that must be protected.

- Do not hard-code secrets into source code.
- Do not commit secrets to Git repositories.
- Do not place sensitive secrets in frontend code that users can inspect.
- Use secret-management systems or protected environment/configuration mechanisms.
- Rotate credentials and revoke compromised keys.
- Use least privilege for service credentials.

KEY IDEA

A secret is not secure merely because it is hidden in a variable name. The storage, access permissions, logging, rotation, and lifecycle all matter.

12. Rate Limiting

Rate limiting restricts how many requests a client, identity, IP, or API key can make during a period.

- Protect login endpoints from credential attacks.
- Protect expensive APIs.
- Reduce abuse and accidental overload.
- Use different limits for different users or endpoints.

```
Too many requests
↓
Rate Limit Check
↓
HTTP 429 Too Many Requests
```

DESIGN TIP

Rate limiting is one layer. Authentication, authorization, input validation, monitoring, and abuse detection still matter.

13. SQL Injection

SQL injection happens when untrusted input is incorrectly incorporated into SQL commands, allowing the input to change the intended query.

X Dangerous idea:
`"SELECT * FROM users WHERE email = " + user_input + """`

✓ Safer approach:

Parameterized query / prepared statement

- Use parameterized queries.
- Use a well-designed ORM/query builder correctly.
- Validate input where appropriate.
- Use database accounts with least privilege.
- Do not rely on escaping alone as the primary defense.

INTERVIEW ANSWER

The strongest basic defense is parameterized queries/prepared statements so user input is treated as data rather than SQL syntax.

14. XSS — Cross-Site Scripting

XSS occurs when attacker-controlled content is executed as script in another user's browser because the application fails to safely handle untrusted content.

- Stored XSS — malicious content is saved and later shown to users.
- Reflected XSS — malicious input is reflected in a response.
- DOM-based XSS — client-side code creates the unsafe behavior.

Defense

- Context-aware output encoding.
- Safe templating frameworks and auto-escaping.
- Careful HTML sanitization when rich HTML is genuinely required.
- Content Security Policy (CSP) as an additional defense.
- Avoid inserting untrusted strings into dangerous DOM contexts.

MEMORY

SQL injection targets the database query. XSS targets the browser's interpretation of untrusted content.

15. CSRF — Cross-Site Request Forgery

CSRF tricks a user's browser into sending an unwanted authenticated request to a site where the user is already signed in.

- Especially relevant to browser-based authentication using automatically attached cookies.
- A malicious site may cause the victim's browser to send a request to the target site.
- The target server may mistake the request for an intentional user action.

Defenses

- CSRF tokens for state-changing requests.

- SameSite cookie settings where appropriate.
- Origin/Referer checks as an additional control when suitable.
- Use appropriate authentication architecture and do not assume that CORS alone is a CSRF defense.

INTERVIEW TIP

CSRF is about abusing the browser's automatic credential sending. XSS is about executing attacker-controlled script/content in the victim's browser.

16. Secrets Management

Secrets management is the controlled lifecycle of sensitive credentials and cryptographic material.

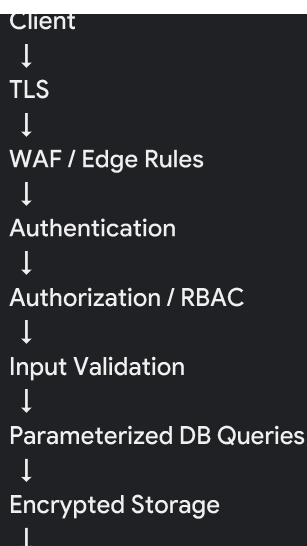
16. Store secrets outside source code.
17. Restrict who/services can read each secret.
18. Encrypt secrets at rest in the secret store.
19. Audit access.
20. Rotate credentials periodically and after suspected compromise.
21. Avoid logging tokens/passwords/API keys.
22. Use separate secrets across environments when practical.
23. Have a revocation/recovery process.

PRODUCTION MINDSET

Treat secrets as short-lived credentials where possible, grant least privilege, and design rotation before you need it.

17. Defense in Depth

Defense in depth means using multiple independent controls so one security failure does not automatically become a complete system compromise.



- No single layer should be treated as perfect.
- Authorization must still be checked even if authentication succeeded.
- Database credentials should have limited privileges.
- Security logs should help detect and investigate suspicious activity.

18. Security Control Cheat Sheet

Threat / Need	Control	Purpose
Identity	Authentication	Verify who the requester is
Permissions	Authorization / RBAC	Control what they can do
Network interception	TLS	Protect data in transit
Stored data exposure	Encryption at rest	Protect stored data
Password theft	Password hashing	Make offline guessing expensive
API abuse	Rate limiting	Control request volume
SQL injection	Parameterized queries	Keep input as data
XSS	Output encoding / sanitization / CSP	Prevent unsafe browser execution
CSRF	CSRF tokens / SameSite / origin checks	Prevent forged browser actions
Credential leakage	Secrets management	Protect and rotate secrets

19. Interview Questions

Q: Authentication vs Authorization?

A: Authentication verifies identity. Authorization decides what an authenticated identity is allowed to do.

Q: What is RBAC?

A: A model where permissions are assigned to roles and users are assigned to roles.

Q: JWT vs session?

A: JWT carries signed claims and can support stateless APIs; sessions keep server-side state and can be centrally revoked. The choice depends on architecture and security requirements.

Q: Is JWT encrypted?

A: Not necessarily. A normal signed JWT provides integrity/authenticity of the token contents, not confidentiality.

Q: What is OAuth 2.0?

A: An authorization framework for delegated access to protected resources. OIDC adds an identity/authentication layer.

Q: Why TLS?

A: To protect data in transit and authenticate the server using certificates.

Q: Why hash passwords instead of encrypting them?

A: Password hashes are designed to be one-way and expensive to brute-force; the application only needs to verify a password, not recover the original.

Q: How prevent SQL injection?

A: Use parameterized queries/prepared statements, safe database access patterns, validation, and least-privilege DB accounts.

Q: XSS vs CSRF?

A: XSS executes attacker-controlled content/script in a user's browser; CSRF abuses automatically attached credentials to forge requests.

Q: How protect secrets?

A: Keep them out of source code, use a secret manager/protected configuration, limit access, rotate, audit, and avoid logging them.

20. Mini System Design Security Exercise

Design security for a student attendance API with students, teachers, and administrators.

24. Use HTTPS/TLS for client-to-server communication.
25. Authenticate users with a suitable mechanism.
26. Use RBAC or fine-grained authorization so students cannot perform teacher/admin operations.
27. Hash passwords with a password-specific hashing algorithm.
28. Protect session cookies or access tokens appropriately.
29. Use parameterized database queries.
30. Validate and safely encode user-controlled content.
31. Protect state-changing browser requests against CSRF where cookie-based authentication makes it relevant.
32. Rate-limit login and sensitive endpoints.
33. Store database credentials and signing keys in a secrets-management solution.
34. Log security-relevant events without logging passwords or tokens.

MASTER QUESTION

For every security feature ask: What threat does it address? Where is the control enforced? What happens if that control fails?

21. One-Page Revision

Concept

Remember

Authentication	Who are you?
Authorization	What are you allowed to do?
RBAC	Permissions grouped into roles
JWT	Signed token carrying claims; not automatically encrypted
Session	Server-side session state referenced by a session ID
OAuth 2.0	Delegated authorization framework
TLS	Protects data in transit
At rest	Protect stored data
Password hashing	Use Argon2id/bcrypt/scrypt + unique salt
API keys	Credentials; protect, scope, rotate
Rate limiting	Control request volume
SQL injection	Prevent with parameterized queries
XSS	Prevent unsafe browser execution
CSRF	Prevent forged authenticated browser requests
Secrets management	Secure storage, access, rotation, auditing
Core memory	Identity → Permissions → Encryption → Validation → Limits → Secrets

PHASE 11 COMPLETE ✓

Secure identity • Secure permissions • Secure data • Secure APIs • Secure secrets